

|                 |                |  |                                       |
|-----------------|----------------|--|---------------------------------------|
| <b>CHALMERS</b> | Anonymous code |  | Consecutive page no<br>Löpande sid nr |
|                 | Anonym kod     |  | Question no.<br>Uppgift nr <b>1</b>   |

## Basic question 1: Hash tables

You find an open addressing hash table (using linear probing and modular compression) in the following state. The tokens **X** are the result of lazy deletions:

|     |   |   |     |   |     |   |   |     |   |
|-----|---|---|-----|---|-----|---|---|-----|---|
| 0   | 1 | 2 | 3   | 4 | 5   | 6 | 7 | 8   | 9 |
| Dog |   | X | Bee | X | Cat |   |   | Ant | X |

For each element, list every number in the range 0–9 that can be its hash value.

### Answer

| Element | Possible hash values (0–9) |
|---------|----------------------------|
| Ant     |                            |
| Bee     |                            |
| Cat     |                            |
| Dog     |                            |

|                 |                |  |                                       |
|-----------------|----------------|--|---------------------------------------|
| <b>CHALMERS</b> | Anonymous code |  | Consecutive page no<br>Löpande sid nr |
|                 | Anonym kod     |  | Question no.<br>Uppgift nr <b>2</b>   |

## Basic question 2: Heaps

Here is a binary max-heap:

|    |    |    |    |    |    |   |   |   |   |
|----|----|----|----|----|----|---|---|---|---|
| 0  | 1  | 2  | 3  | 4  | 5  | 6 | 7 | 8 | 9 |
| 32 | 14 | 20 | 10 | 11 | 16 | 5 | 7 | 5 | 8 |

### Answer

a) Draw the binary max-heap in tree representation.

b) We run the removeMax operation. Draw the resulting heap in tree representation.

|                 |                |  |                                       |
|-----------------|----------------|--|---------------------------------------|
| <b>CHALMERS</b> | Anonymous code |  | Consecutive page no<br>Löpande sid nr |
|                 | Anonym kod     |  | Question no.<br>Uppgift nr <b>3</b>   |

## Basic question 3: graphs

Here is a weighted graph represented as an adjacency matrix.

|   | A | B | C | D | E | F | G |
|---|---|---|---|---|---|---|---|
| A |   | 1 | 2 |   |   |   |   |
| B | 1 |   | 3 |   |   |   |   |
| C | 2 | 3 |   | 5 |   | 1 | 6 |
| D |   |   | 5 |   | 3 |   |   |
| E |   |   |   | 3 | 1 |   | 2 |
| F |   |   | 1 |   |   |   |   |
| G |   |   | 6 |   | 2 |   |   |

- Draw the weighted graph.
- Draw (or highlight) a minimum spanning tree of the graph.

**Lifhack:** If you ever find yourself drawing a graph, things may get less messy if you start by putting a node with lots of edges in the middle.

**Answer**

|                 |                |  |                                       |
|-----------------|----------------|--|---------------------------------------|
| <b>CHALMERS</b> | Anonymous code |  | Consecutive page no<br>Löpande sid nr |
|                 | Anonym kod     |  | Question no.<br>Uppgift nr <b>4</b>   |

## Basic question 4: Merge sort

We do a recursive merge sort of the following 7-element array:

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
| M | E | R | G | I | N | G |

Note:

- Characters are compared according to their position in the alphabet:  $E < G < I < M < N < R$ .
- When splitting a subarray of odd length, the left part should be the larger one.
- The recursive call for the left part happens before the recursive call for the right part.

Answer by listing the merge steps in the order in which they are performed by the algorithm.

Use the following format (example): `merge([A, M], [L, O, S, T]) = [A, L, M, O, S, T]`

**Answer**

|          |                |  |                                       |
|----------|----------------|--|---------------------------------------|
| CHALMERS | Anonymous code |  | Consecutive page no<br>Löpande sid nr |
|          | Anonym kod     |  | Question no.<br>Uppgift nr <b>5</b>   |

## Basic question 5: Complexity

A *streak* in an array is a segment of equal elements. For example, the array [2, 1, 1, 2, 2, 2, 4] has a streak [2, 2, 2] of length 3 (and this is the maximum streak length for this array).

Here is an algorithm (using a helper function) for the maximum streak length:

```

maxStreakLength(array):
    result = 0
    start = 0
    while start < array.length:
        length = streakLength(array, start)
        if result < length:
            result = length
    (1)    start = start + 1
    return result

streakLength(array, start):
    stop = start + 1
    while stop < arr.length and array[start] = array[stop]:
        stop = stop + 1
    return stop - start

```

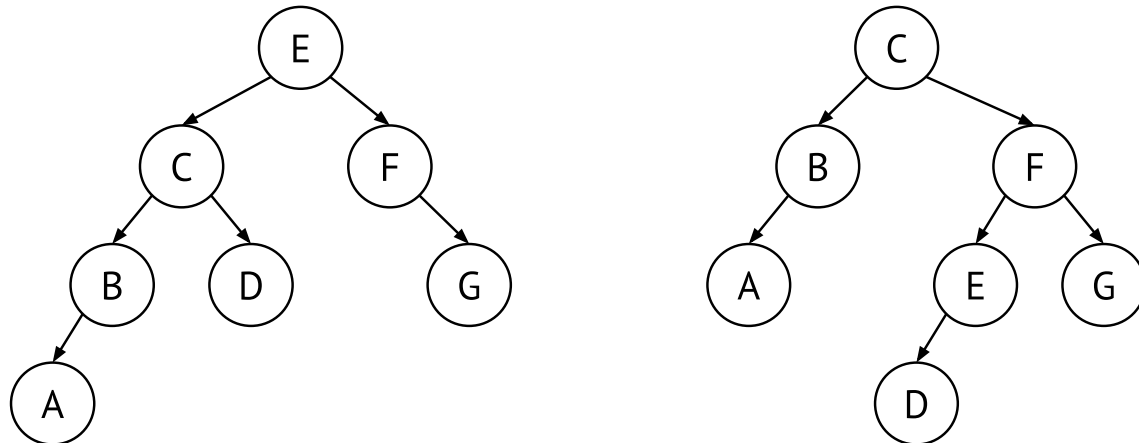
- Determine the asymptotic complexity of maxStreakLength in the length  $N$  of the array.
  - Answer in  $O$ -notation, and as tight and simple as possible.
  - Justify your answer (for example by annotating the above pseudocode).
- We replace line (1) with  $\text{start} = \text{start} + \text{length}$ . What is the asymptotic complexity of maxStreakLength with this modification? (You do not have to justify your answer)

## Answer

|                 |                |  |                                       |
|-----------------|----------------|--|---------------------------------------|
| <b>CHALMERS</b> | Anonymous code |  | Consecutive page no<br>Löpande sid nr |
|                 | Anonym kod     |  | Question no.<br>Uppgift nr <b>6</b>   |

## Basic question 6: Tree rotations

Here are two binary search trees containing the same values:



Transform the left tree into the right tree using **two** tree rotations.

- State the rotations in the order you perform them (where in the tree and in what direction).
- Draw the tree after the first rotation.

**Answer**

|          |                |  |                                       |
|----------|----------------|--|---------------------------------------|
| CHALMERS | Anonymous code |  | Consecutive page no<br>L pande sid nr |
|          | Anonym kod     |  | Question no.<br>Uppgift nr <b>7</b>   |

## Advanced question 7: Set partition

A *partition* of a set is a decomposition into *parts* (disjoint inhabited subsets).<sup>1</sup>

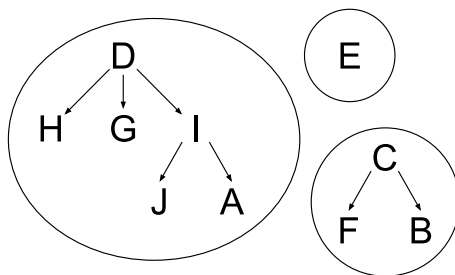
The SetPartition ADT supports the following operations:

```

add(x: Value)                // Add a new element x, forming its own part.
same_part(x: Value, y: Value) → Boolean // Are the elements x and y in the same part?
join(x: Value, y: Value)     // Merge the parts of elements x and y.

```

To efficiently implement this, we represent the elements as nodes of a (rooted) *forest*: a graph that is the disjoint union of (rooted) trees (which represent the parts).<sup>2</sup>



**Example:** a set with 10 elements partitioned into 3 parts (drawn as bubbles), represented as a forest with roots C, D, E

We store the forest using parent pointers (instead of using children pointers as usual):

```

class Node:
    value: Value
    parent: Node // null for a root node
    size: Integer // the size of the tree rooted at this node

class Forest implements SetPartition:
    nodes: Map[Value, Node] = initially empty AVL map

    add(x: Value):
        if x is not in nodes:
            nodes.put(x, new Node(x, null, 1))

```

We impose the following balance invariant: every node has at least twice the size of each of its children. For example, in the picture above, node D has size 6 and its largest child I has size 3.

Implement the missing operations `same_part` and `join`. They should run in  $O(\log(N))$  where  $N$  is the size of the set, assuming values compare in  $O(1)$ . You do not have to justify this complexity.

**Hint:** implement a helper method

```
find_root(x: Value) → Node.
```

<sup>1</sup> Mathematicians call it an *equivalence relation* on the set; the parts are the *equivalence classes*.

<sup>2</sup> Note that the same partition can be represented by different forests.

|          |                |  |                                       |
|----------|----------------|--|---------------------------------------|
| CHALMERS | Anonymous code |  | Consecutive page no<br>Löpande sid nr |
|          | Anonym kod     |  | Question no.<br>Uppgift nr <b>8</b>   |

## Advanced question 8: Game data structure

You got hired as a junior programmer to work on MonsterHorde™. In this game, a player is fighting a (non-empty) horde of enemy monsters. Every monster has a number of *hitpoints* that decrease when the player attacks and increase when a monster is restored by dark magic.

The lead developer decided to store the hitpoints of all the monsters in an array

monsters: Array[Integer]

but gave you the responsibility to efficiently operate on the array:

- attack(x): remove  $x$  hitpoints from the enemy with the most remaining hitpoints.
- restore(x): add  $x$  hitpoints to the enemy with the most remaining hitpoints.
- hasWon() → Boolean: are there ~~any~~ **no** enemies with positive hitpoints left?

Initially, all hitpoint numbers are positive and the array is sorted in descending order (so the monster with the most hitpoints is at position 0). But you do not need to keep the array sorted: all that matters is that it always contains the current hitpoint numbers in some order.

- a) Describe an invariant on the array that helps you implement the operations efficiently.
- b) Implement the three operations (maintaining your invariant, of course).
- c) State the asymptotic complexity of each function in terms of the number  $N$  of enemies.

For full points, two of the operations should be  $O(1)$  and all should be faster than  $O(N)$ .

**Note:** You can freely use operations on data structures and algorithms from the course – you do not have to explain how they work.

### Answer



|          |                |  |                                       |
|----------|----------------|--|---------------------------------------|
| CHALMERS | Anonymous code |  | Consecutive page no<br>Löpande sid nr |
|          | Anonym kod     |  | Question no.<br>Uppgift nr 9          |

## Advanced question 9: Cursed trains

You are given a type  $S$  of *stations* and a type  $T$  of *trains* (both supporting  $O(1)$  comparison).

Each train is cursed to forever cycle through a list of stations:

```
stations_for_train: Map[T, List[S]]           // O(1) get
```

From this, you have already computed which trains stop at a given station:

```
trains_for_station: Map[S, List[T]]           // O(1) get
```

You want to get from station  $A$  to station  $B$  by taking the fewest number of these cursed trains.

Find an algorithm for this that runs in  $O(N \log(N))$  where  $N$  is the total size of the above lists.

**Note:** You can freely use data structures and algorithms from the course – you do not have to explain how they work.

**Hint:** How can you model getting on or getting off a train?

**Answer**